Lab 2
A neural network from scratch

NumPy implementation | Digits classification | Experiment report

1. Data and experiment setup

The load_digits dataset contains 1,797 images with 64 features and 10 classes. Inputs are divided by 16; labels are one-hot encoded. A stratified 70/30 split with random_state=42 gives 1257 training and 540 test examples.

All six configurations use 50 epochs, mini-batches of 32, and plain SGD. Weights are sampled independently from a zero-mean normal distribution with standard deviation 0.01; biases are zero. Initialization seed: 42; shuffle seed: 123. For each width, all learning rates share initial values and batch order. The test set is used only for final evaluation.

2. Network design

64 inputs -> W units -> ReLU -> W units -> ReLU -> 10 outputs -> softmax

Parameter	General shape	W = 16	W = 64
W1 / b1	(64,W) / (W,)	(64,16) / (16,)	(64,64) / (64,)
W2 / b2	(W,W) / (W,)	(16,16) / (16,)	(64,64) / (64,)
W3 / b3	(W,10) / (10,)	(16,10) / (10,)	(64,10) / (10,)
Total parameters	W^2 + 76W + 10	1,482	8,970

3. Forward pass and cross-entropy

For a batch of m examples, X has shape (m,64). Biases broadcast over the batch dimension.

```
Z1 = X @ W1 + b1;       A1 = max(0, Z1)
Z2 = A1 @ W2 + b2;      A2 = max(0, Z2)
Z3 = A2 @ W3 + b3
S  = Z3 - row_max(Z3)
logP = S - log(row_sum(exp(S)))
P = exp(logP)
L = -sum(Y * logP) / m
```

P has shape (m,10). Subtracting the row maximum stabilizes exponentials; computing loss from log-softmax avoids log(0) without clipping. Uniform predictions yield ln(10) = 2.302585.

W=16: initial full-training loss 2.30258552; maximum checked row-sum error 2.22e-16. Shape and tolerance assertions passed.

W=64: initial full-training loss 2.30259249; maximum checked row-sum error 4.44e-16. Shape and tolerance assertions passed.

4. Chain-rule derivation

For one sample, $dL/dP_k = -Y_k/P_k$ and $dP_k/dZ_j = P_k(delta_{kj} - P_j)$. Combining these gives $dL/dZ_j = P_j - Y_j$, since the one-hot target sums to 1. For the batch mean, divide by m. Applying the matrix-product and ReLU chain rules:

```
D3 = (P - Y) / m
dW3 = A2.T @ D3;        db3 = sum_rows(D3)
D2 = (D3 @ W3.T) * (Z2 > 0)
dW2 = A1.T @ D2;        db2 = sum_rows(D2)
D1 = (D2 @ W2.T) * (Z1 > 0)
dW1 = X.T @ D1;         db1 = sum_rows(D1)
```

Every dW and db has its parameter's shape. The bias gradient sums over samples; ReLU uses derivative zero at zero. Update every parameter with theta = theta - learning_rate * gradient.

5. Numerical gradient checking

Each width is checked on its trained learning-rate-0.1 snapshot using the first 32 training examples. This avoids tiny initial gradients that magnify floating-point cancellation. Ten weight entries are sampled uniformly without replacement, seed 2026, with no filtering or resampling. Each weight is perturbed by +/-1e-5 and restored afterward.

g_num = (L(theta + epsilon) - L(theta - epsilon)) / (2 * epsilon).
Relative error = abs(g_num - g_analytic) / max(1e-12, abs(g_num) + abs(g_analytic)).
Passing criterion: relative error < 1e-6 for at least 9 of 10 weights per width.

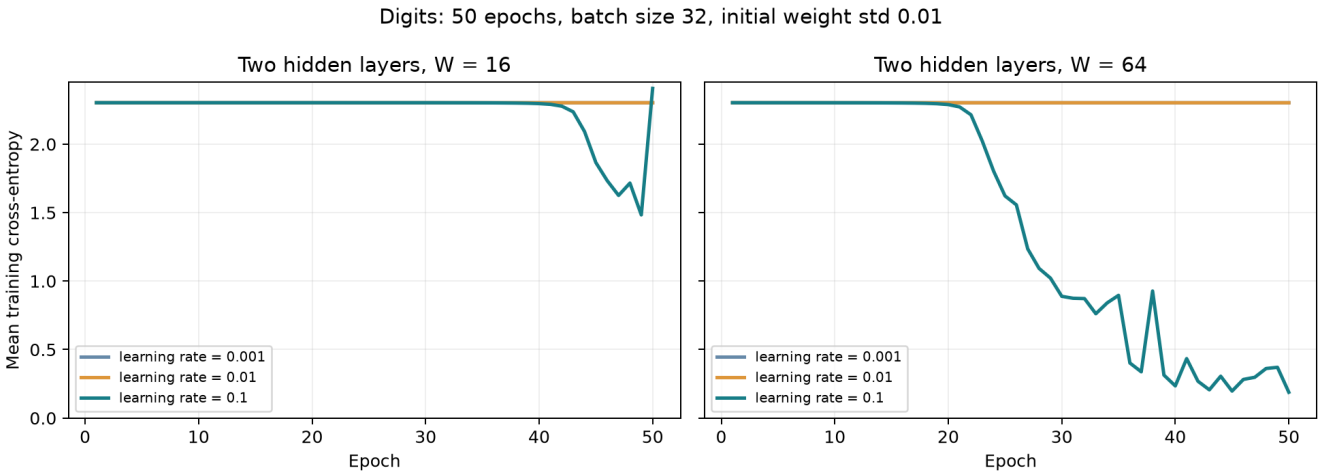
W	Weight [row,col]	Analytical	Numerical	Relative error	Pass
16	W1[2,5]	-2.63630e-02	-2.63630e-02	5.54e-12	Yes
16	W1[31,15]	0.00000e+00	0.00000e+00	0.00e+00	Yes
16	W1[16,0]	0.00000e+00	0.00000e+00	0.00e+00	Yes
16	W1[57,5]	-2.38877e-03	-2.38877e-03	2.74e-09	Yes
16	W2[12,3]	0.00000e+00	0.00000e+00	0.00e+00	Yes
16	W1[32,12]	0.00000e+00	0.00000e+00	0.00e+00	Yes
16	W1[7,2]	0.00000e+00	0.00000e+00	0.00e+00	Yes
16	W1[41,14]	3.64601e-02	3.64601e-02	4.46e-11	Yes
16	W1[33,4]	-3.94118e-02	-3.94118e-02	1.96e-10	Yes
16	W1[57,13]	5.26645e-03	5.26645e-03	2.84e-09	Yes
64	W1[3,41]	1.10220e-02	1.10220e-02	3.71e-10	Yes
64	W1[48,62]	0.00000e+00	0.00000e+00	0.00e+00	Yes
64	W1[24,42]	0.00000e+00	0.00000e+00	0.00e+00	Yes
64	W2[24,15]	9.89889e-05	9.89889e-05	6.41e-09	Yes
64	W2[53,27]	1.45406e-03	1.45406e-03	1.16e-09	Yes
64	W1[50,26]	-6.32602e-05	-6.32602e-05	4.66e-08	Yes
64	W1[11,0]	0.00000e+00	0.00000e+00	0.00e+00	Yes
64	W2[0,29]	0.00000e+00	0.00000e+00	0.00e+00	Yes
64	W1[51,7]	-1.36585e-02	-1.36585e-02	1.89e-10	Yes
64	W2[24,50]	4.00315e-05	4.00315e-05	2.66e-10	Yes

W=16: 10/10 passed. ReLU sign crossings in perturbed passes: 0.

W=64: 10/10 passed. ReLU sign crossings in perturbed passes: 0.

6. Learning-rate and width comparison

W	Learning rate	Final train loss	Train accuracy	Test accuracy
16	0.001	2.302550	10.10%	10.19% (55/540)
16	0.01	2.302479	10.10%	10.19% (55/540)
16	0.1	2.407343	24.90%	23.89% (129/540)
64	0.001	2.302526	10.10%	10.19% (55/540)
64	0.01	2.302344	10.10%	10.19% (55/540)
64	0.1	0.188314	93.16%	90.74% (490/540)



Highest observed test accuracy: 90.74%, W=64, learning rate 0.1. Small initial weights and two ReLU layers make early hidden gradients small, so low learning rates can stay near uniform prediction over 50 epochs. W=16 with learning rate 0.1 has a late loss spike, ending at loss 2.407343 and 23.89% test accuracy, indicating unstable training in that configuration. These single-seed comparisons use the prescribed test split; they are not validation-based model selection or evidence of a statistically reliable ranking.

7. Separate forward/backward timing

Batch size 32; 20 warm-up iterations; median of 100 measurements per pass; one BLAS thread. Pass order alternates. Backward uses a cached forward pass. Data loading, updates, and explicit loss calculation are excluded. Forward includes log-softmax and probabilities.

W	Forward (ms)	Backward (ms)	Backward / forward
16	0.028600	0.028700	1.003
64	0.043250	0.054050	1.250

For W=16, the passes are essentially tied (forward 0.0286 ms; backward 0.0287 ms); this small difference does not establish a meaningful timing advantage. For W=64, the backward pass takes longer (forward 0.0432 ms; backward 0.0541 ms).

Forward uses three matrix products; backward uses five (three weight gradients and two hidden-error products). Forward work is m(64W + W^2 + 10W) multiply-accumulates, versus m(64W + 2W^2 + 20W) backward. At m=32 these are 46,080 / 59,392 for W=16 and 282,624 / 434,176 for W=64. The W-by-W term grows quadratically. Softmax, masks, reductions, allocations, and call overhead explain differences between arithmetic ratios and measured ratios.

Reproducibility: python 3.12.14, numpy 2.5.4, sklearn 1.9.1, matplotlib 3.11.2. Full logs, numerical slopes, environment details, and timing values are included with the notebook.